MTP Speculative Decoding · TurboQuant KV Cache · Apple Silicon

Gemma4 MTP + TurboQuant KV Cache Compression

Version 2: Updated with Live Experimental Results

A Practical Guide for Local Inference on Apple Silicon

89 tok/s	~3.8×	0 retraining	128 GB
Gemma 4 26B	TurboQuant KV compression	Works on any transformer	Unified memory Apple Silicon M5 MacBook Pro Max

Based on hands-on experiments · M5 MacBook Pro Max, 128 GB · macOS 26 · vllm-mlx 0.3.0 nightly

Compiled by **Douglas Mun** with AI assistance

Preface

My original TurboQuant guide (v1, April 2026) focused on the theoretical claims and CUDA-based deployment. This version adds a new chapter — **Gemma 4 MTP (Multi-Token Prediction)** — and updates the Apple Silicon sections with results from hands-on experiments conducted on an M5 MacBook Pro Max in May 2026.

The core finding from these experiments is educational and somewhat counterintuitive: **neither MTP nor TurboQuant improves single-request throughput on Apple Silicon**, even for large models with long context lengths. Understanding *why* is the most valuable thing this guide can teach you. It reveals how Apple Silicon's unified memory architecture is fundamentally different from discrete GPU systems — and changes how you should think about optimising LLM inference on this hardware.

1. Background: The Two Techniques

TurboQuant

TurboQuant (arXiv:2504.19874, ICLR 2026) is a training-free algorithm that compresses the

KV cache — the key-value memory that accumulates during transformer attention — to approximately 3.5 effective bits per value. It achieves up to 6× KV cache memory reduction with near-zero accuracy loss. No retraining. No calibration data. Works on any transformer model out of the box.

It went viral in early 2026 after memory chip stocks dropped on the news: SK Hynix fell 6%, Samsung fell 5%, and Micron fell 3.4%. The implication was that LLMs might need far less memory than previously assumed.

Gemma 4 MTP

MTP (Multi-Token Prediction) is a speculative decoding technique built directly into Gemma 4's model weights. Instead of predicting one token per forward pass, the model drafts several candidate tokens and verifies them simultaneously. If the drafts are correct, multiple tokens are accepted in a single pass — theoretically multiplying throughput.

Critically, Gemma 4 is the first widely available model where the MTP draft head is **embedded in the main model weights**. No separate, smaller drafter model is needed. On vllm-mlx, you enable it with two flags, and it just works.

What This Guide Covers

- How both techniques work mechanically
- Why do they behave differently on Apple Silicon vs CUDA GPUs
- Exact CLI commands and client code for vllm-mlx
- Real benchmark results from this machine
- Every issue encountered and its fix

2. The Memory Wall TurboQuant Targets

Most discussion of LLM memory focuses on model weights. A 70B model in FP16 is ~140 GB — large but fixed. The less-discussed problem is the **KV cache**: it grows dynamically with every token, every layer, and every concurrent user.

The KV Cache Formula

```
KV cache size =
    num_layers
  × sequence_length
  × 2  (keys + values)
  × num_KV_heads
  × head_dim
  × 2 bytes (FP16)
  × batch_size
```

Example — Llama-3.1-70B at 128K context, 10 concurrent users:

```
80 layers × 128,000 tokens × 2 × 8 heads × 128 dim × 2 bytes × 10 users
= ~420 GB
```

This is before model weights. It forces multi-GPU configurations not because the model is too complex to compute, but because the inference memory footprint overflows a single device can handle. The cache grows linearly with context length, layer count, and batch size.

Why Prior Methods Failed

Scalar quantization (FP8, KIVI): The top 1% of KV values are 10–100× larger than the median (outliers). Linear quantization crushes normal token precision to preserve these outliers. KIVI achieved only ~2.6× compression.

Token pruning (SnapKV, PyramidKV): Discards "unimportant" tokens. But you cannot know which tokens will matter later. This introduces irreversible information loss for long-range dependencies.

TurboQuant takes a different approach: it changes the *statistical properties* of the vectors before quantization, rather than trying to work around their bad distribution after the fact.

3. How TurboQuant Works

Rotation + Quantization (PolarQuant)

Step 1 — Random Orthogonal Rotation

Each key/value vector is multiplied by a random orthogonal matrix built from the randomized Hadamard transform. Cost: O(d log d), not O(d^2) — practical at LLM head dimensions of 64–256.

The key property is that a random orthogonal rotation is **norm-preserving**. It doesn't change vector lengths or inner products, so attention dot products remain intact. What it *does* change is how energy is distributed across dimensions.

Step 2 — Beta Distribution Normalization

After rotation, each coordinate independently follows an approximately Beta distribution — regardless of the original distribution. This is the mathematical guarantee that makes TurboQuant work universally: the rotation induces a known, predictable distribution on every coordinate, so a single set of pre-computed quantization codebooks works on any model without calibration.

Step 3 — Pre-computed Lloyd-Max Codebooks

Because the post-rotation distribution is analytically known (Beta), optimal quantization buckets are computed in advance and baked in as constants: no per-model calibration, no dataset, no warm-up.

QJL — Skip It in Production

The paper adds a second stage: Quantized Johnson-Lindenstrauss (QJL), a 1-bit error-correction pass. Community findings from 5+ independent implementation groups:

QJL hurts in practice.

Why: Attention runs dot products through softmax, which exponentially amplifies variance. QJL reduces bias but adds random noise — and softmax magnifies it. MSE-only quantization (biased but lower-variance) wins after softmax.

> **Rule:** Use PolarQuant + MSE-only. Skip QJL for all KV cache applications.

QJL is correct for vector-search use cases without softmax, not for attention.

Compression Sweet Spots

Bit-width	Compression	Quality	Recommendation
4-bit	~3.8×	Indistinguishable from FP16 (≥3B models)	**Production default**
3-bit	~4.9×	Noticeable degradation on models <8B	Large models only
2-bit	~6×	Output degrades severely	Avoid

vLLM Presets (CUDA)

Preset	Keys	Values	Compression	PPL Impact
`turboquant_k8v4`	FP8	4-bit	2.6×	+1.17%
`turboquant_4bit_nc`	4-bit MSE	4-bit	3.8×	+2.71%
`turboquant_k3v4_nc`	3-bit MSE	4-bit	~3.5×	+10.63%
`turboquant_3bit_nc`	3-bit	3-bit	4.9×	+20.59%

> **Apple Silicon note:** These preset names are CUDA-only. vllm-mlx uses a different flag interface — see Section 9.

4. Gemma 4 MTP: Multi-Token Prediction

What MTP Is

Standard autoregressive decoding generates one token per forward pass. For a 26B model, each forward pass loads ~26 GB of weights from memory. At one token per pass, you're doing this hundreds of times per second.

MTP speculative decoding changes this: a lightweight draft head in the model predicts several future tokens in parallel, then the full model verifies them in a single pass. If the drafts are accepted, you get multiple tokens for the cost of one full pass — potentially 2–4× throughput.

What Makes Gemma 4 Special

Most speculative decoding systems require a **separate, smaller model** to act as the drafter (e.g., a 7B drafter for a 70B target). This doubles the download, uses extra memory, and complicates serving.

Gemma 4 embeds the MTP draft head directly in its weights. No separate drafter model exists or is needed. The draft head was trained jointly with the main model, so it has a strong acceptance rate.

```
# Gemma 4 MTP — no separate drafter needed
vllm-mlx serve mlx-community/gemma-4-26b-a4b-it-8bit \
  --enable-mtp \
  --mtp-num-draft-tokens 4 \
  --port 8000
```

The server log confirms it's working:

```
SimpleEngine loaded: mlx-community/gemma-4-26b-a4b-it-8bit
  (MLLM=True, MTP=True(configured=4, effective=4))
```

MTP is Gemma 4 Only

MTP in vllm-mlx is architecture-specific — the draft head must be baked into the weights. Qwen3.6 does not have this. You can use speculative decoding with Qwen using a separate draft model, but there is no small Qwen3 draft model readily available.

MTP Flags Reference

Flag	Default	Notes
`--enable-mtp`	off	Master switch — required
`--mtp-num-draft-tokens 4`	1	Tokens drafted per step; 4 is the recommended default
`--mtp-optimistic`	off	More aggressive acceptance; higher throughput but slightly lower quality

> **Do not use** `--speculative-config '{"method":"mtp",...}'` — that is CUDA vLLM syntax and will error on vllm-mlx.

5. Apple Silicon Architecture: Why the Rules Change

This is the most important section for understanding the benchmark results in Section 6.

Unified Memory

On Apple Silicon, the CPU, GPU, and Neural Engine share the same physical RAM on a single die. There is no PCIe bus. There is no separate GPU VRAM. The M5 Max has ~400 GB/s memory bandwidth between compute units and RAM.

On a discrete GPU system (e.g., H100):

- GPU VRAM: 80 GB, ~3.35 TB/s bandwidth
- System RAM: separate, PCIe-connected
- The bottleneck is often the GPU-VRAM bandwidth

On Apple Silicon:

- Everything shares the same ~400 GB/s bus
- The CPU, GPU, and Neural Engine all compete for the same bandwidth
- There is no separate VRAM to overflow

Why LLM Decoding is Memory-Bandwidth Bound

During autoregressive decoding, each forward pass reads the entire model from memory (26 GB for Gemma 4 26B) to produce a single token. The GPU is doing a tiny amount of arithmetic relative to the massive memory read. This regime is called **memory-bandwidth bound** — the bottleneck is how fast weights can be loaded from DRAM, not how fast the compute units can multiply.

Arithmetic intensity (FLOP/byte) for a 26B model decoding one token:

```
~52 billion FLOPs (2 × params)
÷
~52 GB read (weights, in FP16)
= ~1 FLOP/byte
```

Peak theoretical intensity the GPU *could* sustain: ~100+ FLOP/byte. We're at 1 FLOP/byte. The GPU is idle ~99% of the time waiting for memory.

Why MTP Doesn't Help Here

MTP trades more compute for fewer memory reads — it verifies multiple tokens in one pass. This only helps when:

1. The model is **compute-bound** (GPU is the bottleneck), or
2. There is a **batched workload** (multiple requests sharing a forward pass)

For a single request on Apple Silicon, the GPU is already bandwidth-bound. Adding more compute to verify draft tokens doesn't help — the bottleneck is still waiting for 26 GB of weights loading from RAM.

Why TurboQuant Doesn't Help Throughput Here

TurboQuant reduces KV cache size. At 128 GB of unified memory, a 26B model uses:

- ~26 GB for weights
- ~1–3 GB KV cache at 5K context
- ~99 GB remaining

The KV cache is not the constraint. Reducing it by 4× frees 0.75–2 GB — essentially nothing relative to available RAM. The bottleneck is weight-loading bandwidth, not cache size.

When They DO Help on Apple Silicon

Scenario	MTP helps	TurboQuant helps
Single request, any context	No	No
Multiple concurrent sessions	Yes (if compute-bound)	Yes (cache fits more sessions)
100K+ token contexts	Maybe	Yes (RAM pressure)
16 GB Mac (memory constrained)	No	Yes (prevent OOM)
128 GB Mac, single user	No	No

The real value of TurboQuant on a 128 GB Mac: Fit a second concurrent session, or push context length to 64K+ before hitting memory pressure. Not throughput — headroom.

The real value of MTP on Apple Silicon: Potentially reduces time-to-first-token in multi-request batched scenarios, not tested in these experiments.

6. Live Benchmark Results

All benchmarks: M5 MacBook Pro Max, 128 GB, macOS 26, vllm-mlx 0.3.0 nightly, temperature=0.

6.1 Gemma 4 E4B — MTP Baseline

Model: `mlx-community/gemma-4-e4b-it-bf16` (~15 GB)

Prompt: transformer attention explanation, 3 runs × 512 max_tokens

Config	tok/s
No MTP	46.8
MTP (draft=4)	46.8
MTP + TurboQuant 4-bit	47.4

Observation: No speedup. E4B is 4B parameters — the model is too small for the MTP draft head overhead to be offset by acceptance gains.

6.2 Gemma 4 26B — MTP at 512 Tokens

Model: `mlx-community/gemma-4-26b-a4b-it-8bit` (~26 GB)

Prompt: 44 prompt tokens, 512 max output tokens, 3 runs

Config	tok/s
No MTP	89.2
MTP (draft=4)	88.7

Observation: 89 tok/s for a 26B model is excellent on consumer hardware. MTP adds no throughput. The model is bandwidth-bound.

6.3 Gemma 4 26B — MTP at 2048 Tokens

Prompt: 105 prompt tokens, 2048 max output tokens, 3 runs

Config	tok/s
No MTP	87.5
MTP (draft=4, --mtp-optimistic)	88.1

Observation: Still flat even with `--mtp-optimistic`. Longer output length doesn't change the bottleneck.

6.4 Gemma 4 26B — Long Context Book Summarization

Script: `summarize_books.py` — 5,000 prompt token chunks from Hofstadter PDFs

Source: MarkItDown-extracted text, 2 chunks per book, 514 output tokens each

Book	MTP baseline	MTP + TurboQuant 4-bit	Delta
Gödel, Escher, Bach	69.3 tok/s	69.2 tok/s	0%
Metamagical Themas	68.6 tok/s	69.4 tok/s	+1.2%
Consciousness Explained	68.6 tok/s	69.0 tok/s	+0.6%
Fluid Concepts & Creative Analogies	66.7 tok/s	68.5 tok/s	+2.7%

Observation: Throughput dropped from 89 tok/s (44-token prompt) to 67–69 tok/s (5K-token prompt), confirming KV cache pressure slows decoding at long context lengths. But TurboQuant recovered only 0–3% of that — within noise.

6.5 Qwen3.6-27B — TurboQuant Baseline

Model: `unsloth/Qwen3.6-27B-MLX-8bit` (~35 GB)

Script: `benchmark_qwen.py` — 3 runs × 512 max_tokens, 38 prompt tokens

Config	tok/s
No TurboQuant	15.1
TurboQuant 4-bit	15.4

Observation: Qwen3.6-27B is notably slower (15 tok/s) than Gemma 4 26B (89 tok/s). This is architecture and quantization format — Qwen3.6 uses a more complex MoE variant. TurboQuant again shows no throughput gain.

6.6 Summary Table

Model	Feature	Context	Before	After	Gain
Gemma 4 E4B	MTP	512 tok	46.8	46.8	0%
Gemma 4 E4B	TurboQuant	512 tok	46.8	47.4	+1.3%
Gemma 4 26B	MTP	512 tok	89.2	88.7	0%
Gemma 4 26B	MTP	2048 tok	87.5	88.1	0%
Gemma 4 26B	TurboQuant	5K prompt	67–69	68–69	+0–3%
Qwen3.6-27B	TurboQuant	512 tok	15.1	15.4	+2%

Definitive conclusion: On M5 Max with 128 GB unified memory at single-request workloads, neither MTP nor TurboQuant improves tok/s. Both are headroom features — they matter for multi-user concurrency, very long contexts, or memory-constrained machines. Not single-request speed.

7. What Actually Works on vllm-mlx

Core Inference

- `vllm-mlx serve` + HTTP API — fully functional
- `/v1/chat/completions` — correct endpoint for Gemma 4
- `/v1/completions` — correct for Qwen3.6 (with chat template embedded in prompt)
- `temperature`, `top_p`, `max_tokens`, `stop` — all work
- Default port: **8000** (not 8001 — common assumption mistake)

MTP (Gemma 4 only)

- `--enable-mtp --mtp-num-draft-tokens 4` — flags accepted, MTP active
- `--mtp-optimistic` — accepted, no errors
- MTP head built into Gemma 4 weights — **no separate drafter download needed**
- Confirmed in startup log: `MTP=True(configured=4, effective=4)`
- Does NOT provide throughput speedup at single-request scale on Apple Silicon

TurboQuant KV Cache

- `--kv-cache-quantization --kv-cache-quantization-bits 4` — works on Gemma 4 and Qwen3.6
- 4-bit recommended; 3-bit degrades quality on models <8B
- No confirmation log line — flags accepted silently; no error = active
- Does NOT provide throughput speedup at single-request scale on Apple Silicon

Tool Calling (Gemma 4)

- `--enable-auto-tool-choice --tool-call-parser gemma4` — works
- Returns proper OpenAI-format `tool_calls` JSON array
- `finish_reason: "tool_calls"` correctly signals the model wants to call a tool
- **Limitation:** E4B (4B params) ignores tool results in Turn 2 — too small to ground multi-turn tool use. 26B is expected to work correctly.
- **Quirk:** `content` field may contain thinking trace alongside `tool_calls` even with `"thinking": false` — strip client-side when `finish_reason == "tool_calls"`

Vision (Gemma 4)

- No extra flags needed — vision encoder loads automatically
- `model_type: mllm` in `/health` endpoint confirms vision is active
- Send images as base64: `{encoded}`
- Correctly identifies and describes images, including CLI screenshots

Qwen3.6-27B

- `--reasoning-parser qwen3` — required for proper `<think>` block handling
- Suppress visible reasoning: embed empty `<think>\n\n</think>` in your prompt
- Strip any leaked `<think>` blocks: `re.sub(r'<think>.*?</think>', '', text, flags=re.DOTALL)`

8. Issues Encountered and Fixes

Issue 1: `pip install vllm` fails — wrong package

Error: `externally-managed-environment` or cmake/CUDA build errors

Cause: `vllm` (the CUDA package) is not installable on macOS

Fix: Install `vllm-mlx` with the nightly flag:

```
python3 -m pip install vllm-mlx --pre --extra-index-url
https://wheels.vllm.ai/nightly
```

Issue 2: Default port is 8000, not 8001

Symptom: `Connection refused` when hitting port 8001

Fix: vllm-mlx defaults to `8000`. Either omit `--port` or pass `--port 8000` explicitly.

Issue 3: Gemma 4 thinking leak

Symptom: `<|channel>thought...<channel|>` in model output

Fix: Add `"thinking": false` to the request JSON. Only works on `/v1/chat/completions`.

```
requests.post(BASE, json={
    "model": MODEL,
    "thinking": False,  # suppresses chain-of-thought output
    "messages": [...]
})
```

Issue 4: zsh intercepts `@filename` in curl

Symptom: `zsh: no matches found: @payload.json`

Cause: zsh glob expansion treats `@file` as a glob in curl's `-d @file`

Fix: Use Python `requests` with the `json=` parameter instead of curl for file-based payloads. Or add `alias curl='noglob curl'` to `~/.zshrc`.

Issue 5: macOS screenshot filenames have invisible Unicode characters

Symptom: `FileNotFoundError` when hardcoding screenshot paths

Cause: macOS inserts a narrow no-break space ( ) before AM/PM in screenshot filenames

Fix: Never hardcode screenshot paths. Use:

```
import glob
img_path = glob.glob('/path/to/dir/*.png')[0]
```

Issue 6: `eval()` in tool execution is a security risk

Symptom: Code review flags arbitrary code execution

Cause: Model-provided text passed directly to `eval()` can execute any Python code

Fix: Use `ast.literal_eval()` — evaluates only Python literals, not arbitrary expressions:

```
# Wrong
result = eval(args["expression"])
# Right
import ast
result = ast.literal_eval(args["expression"])
```

Issue 7: Benchmark average calculation

Symptom: Inflated average tok/s when averaging per-run rates

Cause: Averaging rates (1/time) is not the same as total throughput

Fix: Sum tokens and time, then divide:

```
# Wrong
avg = sum(toks/t for toks, t in zip(token_list, elapsed_list)) / runs
# Right
avg = sum(token_list) / sum(elapsed_list)
```

Issue 8: `pip` binary broken in venv

Symptom: `ModuleNotFoundError: No module named 'pip'` when running `pip install`

Fix: Always use `python3 -m pip install` instead of `pip install`:

```
python3 -m pip install markitdown[pdf]
```

Issue 9: pypdf extracts incomplete text from PDFs

Symptom: Missing chapters, blank pages, no structural markup

Fix: Use MarkItDown, which preserves headings, structure, and extracts more content:

```
python3 -m pip install "markitdown[pdf]"
from markitdown import MarkItDown
md = MarkItDown()
text = md.convert("book.pdf").text_content
```

I Am A Strange Loop: pypdf extracted ~254K tokens; MarkItDown extracted ~296K tokens with full chapter structure.

Issue 10: CUDA-only flags silently fail or error

These flags are not valid on Apple Silicon:

Flag	Platform	What to use instead
`--kv-cache-dtype turboquant_4bit_nc`	CUDA only	`--kv-cache-quantization --kv-cache-quantization-bits 4`
`--speculative-config '{"method":"mtp"}'`	CUDA vLLM only	`--enable-mtp --mtp-num-draft-tokens 4`
`--gpu-memory-utilization 0.95`	CUDA only	Remove — no equivalent

Issue 11: vllm-mlx has no Python API

Symptom: `ImportError: cannot import name 'LLM' from 'vllm'`

Cause: vllm-mlx is server-only — there is no `LLM()` Python class

Fix: Always use `vllm-mlx serve` + HTTP requests:

```
# Wrong
from vllm import LLM
llm = LLM(model="gemma-4...", device="mlx")
# Right
import requests
```

```
r = requests.post("http://localhost:8000/v1/chat/completions", json={...})
```

9. Complete Setup Guide

Prerequisites

Verify your terminal is running natively on arm64 — not under Rosetta 2:

```
arch   # must return: arm64
```

Step 1: Ensure arm64 Homebrew

Apple Silicon Macs can have two Homebrew installations. You need arm64:

Homebrew	Location	Architecture
Intel (Rosetta)	`/usr/local`	x86_64 — wrong
Apple Silicon	`/opt/homebrew`	arm64 — correct

Add to `~/.zprofile`:

```
if [[ -x /opt/homebrew/bin/brew ]]; then
    eval "$(/opt/homebrew/bin/brew shellenv)"
fi
```

Then: `source ~/.zprofile && which brew` — must show `/opt/homebrew/bin/brew`.

Step 2: Install arm64 Python 3.12

```
brew install python@3.12
# Verify it is arm64
file $(brew --prefix python@3.12)/bin/python3.12
# Must say: Mach-O 64-bit executable arm64
```

If it says `x86_64`, your Homebrew PATH is still wrong — return to Step 1.

Step 3: Create the Virtual Environment

```
# Always use the explicit brew-managed Python path
$(brew --prefix python@3.12)/bin/python3.12 -m venv ~/Develop/vllm-mlx
source ~/Develop/vllm-mlx/bin/activate
# Verify before installing anything
arch                          # arm64
python --version              # Python 3.12.x
python -m pip debug --verbose 2>/dev/null | grep arm64 | head -1
# Must show: cp312-cp312-macosx_XX_0_arm64
```

Step 4: Install vllm-mlx Nightly

The stable PyPI release lacks `--kv-cache-quantization`. The nightly is required.

```
python3 -m pip install vllm-mlx --pre --extra-index-url
https://wheels.vllm.ai/nightly
```

Verify flags available in your build:

```
vllm-mlx serve --help | grep -i "cache"
vllm-mlx serve --help | grep -i "mtp"
```

Step 5: Serve Models

Gemma 4 E4B — development/testing:

```
vllm-mlx serve mlx-community/gemma-4-e4b-it-bf16 \
  --enable-mtp \
  --mtp-num-draft-tokens 4 \
  --port 8000
```

Gemma 4 26B — best quality + MTP:

```
vllm-mlx serve mlx-community/gemma-4-26b-a4b-it-8bit \
  --enable-mtp \
  --mtp-num-draft-tokens 4 \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --port 8000
```

Gemma 4 26B with tool calling:

```
vllm-mlx serve mlx-community/gemma-4-26b-a4b-it-8bit \
  --enable-mtp \
  --mtp-num-draft-tokens 4 \
  --enable-auto-tool-choice \
  --tool-call-parser gemma4 \
  --port 8000
```

Qwen3.6-27B with TurboQuant:

```
vllm-mlx serve unsloth/Qwen3.6-27B-MLX-8bit \
  --kv-cache-quantization \
  --kv-cache-quantization-bits 4 \
  --port 8001 \
  --reasoning-parser qwen3
```

Step 6: Verify the Server

```
curl http://localhost:8000/health
# {"status":"healthy","model_loaded":true,"model_name":"...","model_type":"mllm"}
# model_type: mllm = vision active, llm = text-only
```

Recommended Models

Model	Size	Use
`mlx-community/gemma-4-e4b-it-bf16`	~15 GB	Fast iteration, development
`mlx-community/gemma-4-26b-a4b-it-8bit`	~26 GB	Best quality + MTP
`mlx-community/gemma-4-31b-it-8bit`	~34 GB	Maximum quality
`unsloth/Qwen3.6-27B-MLX-8bit`	~35 GB	TurboQuant baseline, reasoning

> Use `mlx-community/` HuggingFace IDs for Gemma 4 — not `google/`. The Google IDs point to the original weights without MLX quantization.

10. Client Code Examples

Basic Chat — Gemma 4

```
import requests

BASE = "http://localhost:8000/v1/chat/completions"
MODEL = "mlx-community/gemma-4-26b-a4b-it-8bit"

def chat(prompt, max_tokens=1024):
    r = requests.post(BASE, json={
        "model": MODEL,
        "thinking": False,      # suppress chain-of-thought output
        "temperature": 0.7,
        "top_p": 0.9,
        "max_tokens": max_tokens,
        "messages": [{"role": "user", "content": prompt}]
    })
    r.raise_for_status()
    return r.json()["choices"][0]["message"]["content"]

print(chat("Explain transformer attention in plain English."))
```

Vision — Send a Local Image

```
import requests, base64, glob, mimetypes, pathlib

BASE = "http://localhost:8000/v1/chat/completions"
MODEL = "mlx-community/gemma-4-26b-a4b-it-8bit"

def ask_about_image(img_path, question):
    with open(img_path, "rb") as f:
        img_b64 = base64.b64encode(f.read()).decode()
    mime = mimetypes.guess_type(img_path)[0] or "image/png"

    r = requests.post(BASE, json={
        "model": MODEL,
        "thinking": False,
        "temperature": 0.3,
        "max_tokens": 512,
        "messages": [{"role": "user", "content": [
            {"type": "image_url",
             "image_url": {"url": f"data:{mime};base64,{img_b64}"}},
            {"type": "text", "text": question}
        ]}]
    })
    r.raise_for_status()
    return r.json()["choices"][0]["message"]["content"]

# Use glob to handle macOS narrow no-break space in screenshot filenames
```

```
img = glob.glob(str(pathlib.Path(".") / "*.png"))[0]
print(ask_about_image(img, "What is in this screenshot?"))
```

Tool Calling — Two-Turn Loop

```
import ast, json, datetime, requests

BASE = "http://localhost:8000/v1/chat/completions"
MODEL = "mlx-community/gemma-4-26b-a4b-it-8bit"
TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "run_python",
            "description": "Evaluate a Python expression and return the result",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {"type": "string",
                                   "description": "A Python expression to
evaluate"}
                },
                "required": ["expression"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_time",
            "description": "Return the current date and time",
            "parameters": {"type": "object", "properties": {}}
        }
    }
]

def dispatch(name, args):
    if name == "run_python":
        # ast.literal_eval is safe — evaluates literals only, not arbitrary code
        return str(ast.literal_eval(args["expression"]))
    if name == "get_time":
        return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    return f"Unknown tool: {name}"

def run_tool_loop(user_prompt):
    messages = [{"role": "user", "content": user_prompt}]

    # Turn 1: let model decide whether to call a tool
    r = requests.post(BASE, json={
        "model": MODEL, "thinking": False, "temperature": 0,
        "tools": TOOLS, "tool_choice": "auto", "messages": messages
    })
    r.raise_for_status()
    msg = r.json()["choices"][0]["message"]
```

```
    if r.json()["choices"][0]["finish_reason"] != "tool_calls":
        return msg.get("content", "")

    # Execute each tool call
    messages.append(msg)
    for call in msg["tool_calls"]:
        try:
            args = json.loads(call["function"]["arguments"])
        except json.JSONDecodeError:
            args = {}
        result = dispatch(call["function"]["name"], args)
        messages.append({
            "role": "tool",
            "tool_call_id": call["id"],
            "content": str(result)
        })

    # Turn 2: model synthesises the tool results
    r2 = requests.post(BASE, json={
        "model": MODEL, "thinking": False, "temperature": 0,
        "tool_choice": "none", "messages": messages
    })
    r2.raise_for_status()
    return r2.json()["choices"][0]["message"].get("content") or "(no response)"

print(run_tool_loop("What is 1234 * 5678? Also, what time is it?"))
```

Qwen3.6 Chat (Completions Endpoint)

```
import re, requests

BASE = "http://localhost:8001/v1/completions"
MODEL = "unsloth/Qwen3.6-27B-MLX-8bit"

def qwen(prompt, max_tokens=1024):
    # Embed empty <think> block to suppress visible reasoning
    formatted = (
        f"<|im_start|>user\n{prompt}<|im_end|>\n"
        f"<|im_start|>assistant\n<think>\n\n</think>\n\n"
    )
    r = requests.post(BASE, json={
        "model": MODEL,
        "prompt": formatted,
        "max_tokens": max_tokens,
        "temperature": 0,
        "stop": ["<|im_end|>", "<|im_start|>"]
    })
    r.raise_for_status()
    text = r.json()["choices"][0]["text"]
    # Strip any leaked thinking blocks
    return re.sub(r"<think>.*?</think>\s*", "", text, flags=re.DOTALL).strip()
```

```
print(qwen("Explain the Hofstadter strange loop concept in two paragraphs."))
```

Shell Aliases for Daily Use

Add to `~/.zshrc`:

```
# Gemma 4 — chat completions, thinking suppressed
gemma() {
  curl --fail-with-body -sS --connect-timeout 5 --max-time 300 \
    "http://127.0.0.1:8000/v1/chat/completions" \
    -H "Content-Type: application/json" \
    -d "$(jq -n --arg p "$*" \
      '{model:"mlx-community/gemma-4-26b-a4b-it-8bit",
        max_tokens:1024, temperature:0.7, top_p:0.9, thinking:false,
        messages:[{role:"user",content:$p}]}')" \
    | jq -r '.choices[0].message.content'
}

# Qwen3.6 — completions with chat template, strips <think> blocks
qwen() {
  curl --fail-with-body -sS --connect-timeout 5 --max-time 300 \
    "http://127.0.0.1:8001/v1/completions" \
    -H "Content-Type: application/json" \
    -d "$(jq -n --arg p "$*" \
      '{model:"unsloth/Qwen3.6-27B-MLX-8bit",max_tokens:1024,temperature:0,

prompt:"<|im_start|>user\n"+$p+"<|im_end|>\n<|im_start|>assistant\n<think>\n\n</t
hink>\n\n",
        stop:["<|im_end|>","<|im_start|>"]}')" \
    | jq -r '.choices[0].text' \
    | perl -0pe "s/<think>.*?<\/think>\s*//gs"
}
```

11. When to Use Each Feature

Use MTP when:

- Running Gemma 4 (it's free — built into the weights, always enable it)
- Serving multiple concurrent users where batching can be exploited
- Latency-to-first-token matters more than steady-state throughput

Use TurboQuant when:

- You need context lengths beyond 32K tokens
- Running multiple concurrent sessions (extends how many fit in memory)
- Memory pressure is causing macOS swap (visible in Activity Monitor)
- On a machine with 16–24 GB unified memory where the model barely fits

Don't expect either to improve throughput when:

- Running single-user, single-request workloads on M-series Mac
- Using short-to-medium context (under 32K tokens)
- The machine has abundant RAM (128 GB), and the model fits comfortably

Measuring Memory Impact

TurboQuant's real benefit shows up in memory, not tok/s. To measure:

```
# Before and after enabling --kv-cache-quantization
vm_stat | grep "Pages active\|Pages wired\|Pages occupied"

# GPU/ANE power (shows thermal + compute pressure)
sudo powermetrics --samplers gpu_power -i 1000 -n 5
```

Or: Activity Monitor > Memory tab — watch "Memory Used" and "Swap Used" while running long conversations.

12. Key References

Resource	URL
TurboQuant paper (ICLR 2026)	arXiv:2504.19874
HIGGS (mathematical foundation)	arXiv:2411.17525
KIVI (prior KV cache method)	arXiv:2402.02750
vLLM upstream PR #38479	github.com/vllm-project/vllm/pull/38479
Community validation suite	github.com/0xSero/turboquant
turbokv Python package	github.com/vivekvar-dl/turboquant
Google Research blog	research.google/blog/turboquant
Gemma 4 HuggingFace collection	huggingface.co/collections/google/gemma-4
mlx-community models	huggingface.co/mlx-community
vllm-mlx nightly wheels	wheels.vllm.ai/nightly
